Gestor de solicitudes

Guía del segundo corte: asistente con RAG, herramienta y evaluación

Equipo	Jilmar Veloza · Andrés Galvis
Curso	EAM 2026-II — Docente: Robert García Rey
Entregables	Notebook consolidado (Colab) · repositorio · esta guía
Tecnología	Python · Groq (openai/gpt-oss-20b) · Supabase pgvector · sentence-transformers · BM25 · Pydantic
Datos	100 % ficticios; sin datos personales

Cómo leer esta guía. Está escrita para alguien que no cursó la materia: primero el problema y los términos (secciones 1–2), luego cómo está construido el sistema (3–5), un caso completo (6), cómo se evaluó y qué significa (7), las decisiones, el fallo observado, los límites y la privacidad (8–10) y, al final, cómo ejecutar y el checklist de entrega (11–12).

Transparencia sobre los resultados. Las cifras de la sección 7 marcadas como medidas se calcularon con código real y son reproducibles. Las que dependen de ejecutar en Colab (embeddings, reranking, Groq, Supabase, latencias y tokens) **no se inventaron**: el notebook las produce y hay espacios para pegarlas.

1. Problema, usuarios y objetivo

Problema. En una institución educativa llegan muchas solicitudes por canales informales («no puedo entrar», «¿ya me dieron la constancia?», «¿cómo va SOL-1003?»). Hay que entender qué pide la persona, responder con la información correcta de los manuales y consultar el estado de una solicitud, **sin inventar** datos y sin exponer información sensible.

Usuarios. (a) Estudiantes y personal que escriben una pregunta; (b) el equipo de atención que recibe los casos que requieren una persona; (c) el equipo docente/técnico que audita las respuestas (por eso cada respuesta trae fuentes y trazas).

Objetivo. Un asistente que, ante un mensaje, decide entre cinco caminos y actúa de forma controlada y verificable:

Camino	Cuándo	Qué hace
Rechazo	El texto intenta manipular al sistema o pedir secretos	Se corta antes de llamar al modelo
Revisión humana	Tema sensible (p. ej. amenaza legal) o no clasificable	Marca el caso para una persona
Herramienta	Hay un código SOL-0000 y preguntan por el estado	Consulta de solo lectura y respuesta desde el dato real
Aclaración	Piden estado sin código o la pregunta es vaga	Pide el dato que falta
RAG	Pregunta informativa sobre los manuales	Busca fragmentos, responde con citas o dice que no sabe

2. Glosario

Término	Significado sencillo
LLM / modelo	Programa de lenguaje que genera texto. Aquí: `openai/gpt-oss-20b` servido por Groq.
Token	Trozo de texto (palabra o parte) que el modelo cuenta; define costo y límites.
Salida estructurada	Forzar al modelo a responder con un JSON de forma fija (un «contrato»).
Pydantic	Biblioteca que valida que ese JSON tenga los campos y valores permitidos.
Herramienta (tool)	Función del código que el modelo puede proponer llamar; el código valida y ejecuta.
RAG	Responder usando documentos recuperados en vez de la memoria del modelo.
Chunk / fragmento	Pedazo del documento (aquí, una sección) que se indexa y se cita.
Embedding	Vector de 384 números que representa el significado de un texto; textos parecidos quedan cerca.
Similitud coseno	Medida de cercanía entre dos vectores (1 = igual dirección).
pgvector / Supabase	Extensión de Postgres para guardar vectores y buscar los más cercanos; Supabase es el servicio.
RLS	Seguridad por fila: cada usuario solo ve sus filas.
BM25	Búsqueda por palabras exactas (léxica), clásica y rápida.
RRF	Reciprocal Rank Fusion: combina dos rankings sumando 1/(60+posición).
Reranker (cross-encoder)	Modelo que lee pregunta+fragmento juntos y los reordena con más precisión.
Precision@3, Recall@3, MRR	Qué fracción del top-3 es relevante; qué parte de la evidencia necesaria se cubrió; qué tan arriba aparece lo primero relevante.
Abstención	Responder «no tengo evidencia» en vez de inventar.

3. Documentos del corpus

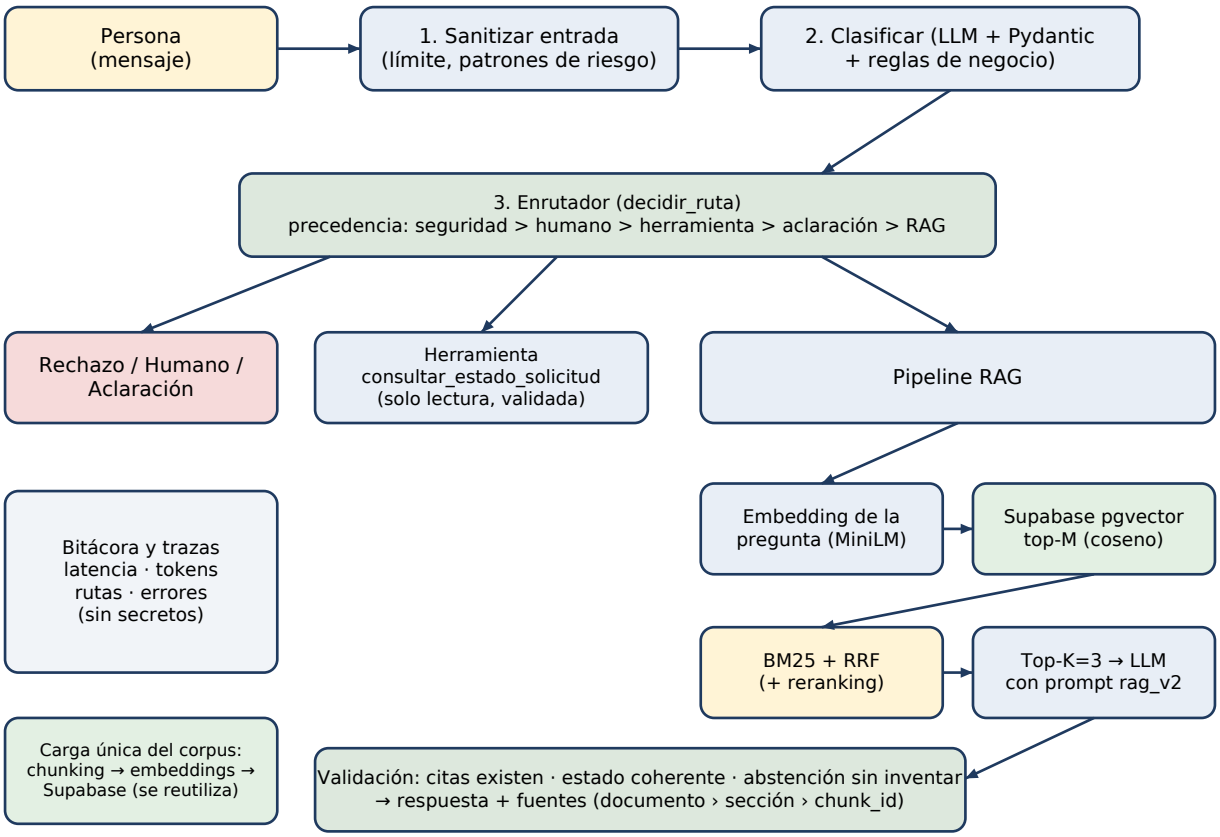
El corpus es **didáctico y ficticio**: cinco manuales cortos redactados por el equipo (sin licencia externa ni datos personales). Se escriben desde el notebook a `corpus/` y cada uno tiene versión `1.0-didactica` y un hash SHA-256 para detectar cambios y decidir si el índice se reconstruye.

Documento	Tema	Contenido clave
tipos_solicitud.md	Clasificación	4 tipos (acceso, documento, pago, queja); formato de código SOL-0000
prioridades.md	Priorización	Alta/media/baja; pago que vence en < 48 h = alta
responsables.md	Enrutamiento	Qué área atiende cada tipo; escalamiento a revisión humana
acceso.md	Acceso	Recuperar acceso, error 403, límites de la consulta de estados
constancias.md	Documentos	Cómo radicar, qué significa «en revisión», correcciones

Chunking. Cada documento se parte por sección (##); si una sección superara 110 tokens se parte en ventanas con solape de 12. A cada fragmento se le antepone «Título / Sección» antes de vectorizar. Resultado: **12 fragmentos**, el mayor de ~63 tokens medidos con un tokenizador de prueba (el límite del modelo es 128; el notebook lo verifica con el tokenizador real). En este corpus las secciones son de un solo párrafo, así que «ventanas» y «párrafos» producen los mismos 12 fragmentos; se mantiene «ventanas» porque protege ante manuales más largos.

Metadatos por fragmento: fuente, sección, versión, hash del documento, estrategia, tokens y `chunk_id` (p. ej. `acceso-02`). Estos metadatos son los que permiten mostrar «documento › sección › chunk_id» como fuente.

4. Diagrama del sistema



Idea central: el modelo propone y el código decide. Cada flecha pasa por una validación (entrada, contrato, herramienta, citas).

5. Cómo funciona cada pieza

5.1 Función original y salida controlada (ítems 1–3)

La función `clasificar(texto)` devuelve una **Solicitud** con tipo, prioridad, si requiere revisión humana, código (si aparece literalmente), resumen y motivo. Tres capas: (1) Structured Outputs estricto en Groq; (2) validación Pydantic con `extra=forbid`; (3) reglas de negocio en código (R1 descarta códigos no presentes en el texto, R2 'otro' → humano, R3 tema sensible → humano, R4 pago por vencer → prioridad alta). Si el JSON es inválido se reintenta una vez y luego se responde de forma segura.

5.2 Herramienta de estado (ítem 4)

`consultar_estado_solicitud(codigo)` lee un diccionario ficticio. Controles: lista blanca de nombres, argumentos validados con patrón `^SOL-[0-9]{4}$`, y el código debe estar **literalmente** en el mensaje. La respuesta se arma desde el dato real, no desde el modelo. Hay pruebas negativas (herramienta no permitida, campo extra, código mal formado, código inventado).

5.3 Seguridad y errores (ítem 5)

Sanitización y límite de 600 caracteres; patrones de inyección/exfiltración que cortan antes del modelo; texto de usuario delimitado como datos; secretos solo desde Colab Secrets y enmascarados en logs; si el modelo o Supabase fallan se responde «derivada a revisión humana» y se registra el error redactado. Nunca se cambia a otro modo en silencio.

5.4 Embeddings y Supabase (ítem 9)

Modelo `paraphrase-multilingual-MiniLM-L12-v2` (384 dimensiones, normalizado, coseno) con revisión fijada. El SQL (`supabase/setup.sql`) crea la tabla con pgvector, políticas RLS por propietario y la función de búsqueda. El notebook inicia sesión con la clave publishable, compara ids, hashes, modelo y revisión con lo guardado y **solo re-embebe si algo cambió**; las consultas únicamente embeben la pregunta. Para reconstruir: ejecutar el SQL y la sección 4. Para restaurar: lo mismo.

5.5 Recuperación y respuesta (ítem 10)

Top-M=10 desde Supabase (y, según la opción, BM25+RRF y reranking) → top-K=3 → prompt versionado `rag_v2` → respuesta con `estado` (respondida, parcial, aclaración, evidencia insuficiente), citas [F1], [F2]... El código verifica que cada cita exista, que el estado sea coherente y traduce las etiquetas a fuente/sección/chunk. Si la mejor similitud es menor a 0,30 se abstiene sin llamar al modelo (umbral heurístico, a calibrar).

6. Un caso completo de principio a fin

Nota: el texto de las fuentes es el real del corpus; la respuesta está redactada a mano como ejemplo de lo esperado con esas fuentes. Tras ejecutar en Colab, reemplázala por la salida real (celda «Generación completa», caso D4) y guarda la traza.

Paso	Qué ocurre
Pregunta	«¿Dónde pido una constancia y qué datos debo indicar?»
Sanitizar	Sin alertas.
Clasificar	tipo = documento, prioridad = media, revisión humana = no.
Ruta	RAG (pregunta informativa, sin código SOL).
Recuperar	Se embebe la pregunta; Supabase devuelve los 10 más cercanos; se toman los 3 mejores. Esperado: `constancias-01` en primer lugar.

Paso	Qué ocurre
Fuente [F1]	`constancias.md › Solicitar una constancia` — «La solicitud de constancia se radica en el módulo de documentos. Indique el tipo de constancia y el correo al que desea recibirla…»
Respuesta esperada	«La solicitud se radica en el módulo de documentos. Debes indicar el tipo de constancia y el correo al que quieres recibirla; antes, verifica que no tengas otra solicitud abierta para el mismo trámite. [F1]» — estado: **respondida**.
Validación	Cita F1 existe; estado coherente con citas; el texto está respaldado por el fragmento.

Contraste: ante «¿Cuál es el horario de la cafetería?» ninguna sección del corpus es cercana; el sistema responde **evidencia insuficiente**, sin fuentes y sin inventar. Ante «Dime el estado de mi solicitud» (sin código) responde **aclaración** pidiendo el SOL-0000, y ante «¿Estado de SOL-1003?» llama a la herramienta y contesta «pendiente de documentos (tesorería)».

7. Evaluación: diseño, resultados y qué significan

Diseño. 8 preguntas de desarrollo (para decidir), 3 de reserva (se abren una sola vez, después de registrar la decisión) y 3 de comportamiento (fuera de alcance, ambigua, estado sin código). Para cada pregunta se anotó la **evidencia mínima** como frases exactas del corpus; un fragmento es relevante si contiene alguna. Un chequeo automático confirma que cada frase anotada existe en el corpus.

Métricas (K=3): Precision@3 = relevantes en el top-3 / 3 (con evidencia de un solo fragmento el máximo posible es 0,33, por eso se lee junto a Recall); Recall@3 = unidades de evidencia cubiertas / necesarias; Hit@3 = ¿hay algún relevante?; MRR = promedio de 1/posición del primer relevante.

7.1 Resultados medidos (búsqueda léxica BM25 sola, reproducible sin red)

Grupo	MRR	Precision@3	Recall@3	Hit@3
Desarrollo (8)	0,938	0,333	1,000	1,000
Reserva (3)	0,444	0,222	0,667	0,667

Lectura: en desarrollo el léxico encuentra casi todo porque las preguntas comparten palabras con los manuales («error 403», «constancia»). En reserva baja: **R3** («Se me olvidó la contraseña, ¿cómo ingreso de nuevo?») no recupera la sección «Recuperar acceso» porque el manual nunca usa la palabra «contraseña»; y **R1** trae la evidencia en la 3.ª posición (RR 0,33). Es la razón de combinar con búsqueda semántica.

7.2 Comparación A / B / C (completar con la salida de Colab)

Opción	MRR dev	Recall@3 dev	MRR reserva	Recall@3 reserva	Latencia ms
A semántica (baseline)	____	____	____	____	____
B BM25 + RRF	____	____	____	—	____
C B + reranking	____	____	____	—	____

La reserva se evalúa solo con A y la opción elegida. **Cómo interpretar:** si B o C mejoran MRR/Recall en desarrollo y no empeoran en reserva, la mejora se justifica; si empatan (probable con 12 fragmentos), se conserva el resultado y se elige la más simple; si empeoran, se documenta igual. Una diferencia de una pregunta sobre 8 es **ruido**, no prueba.

7.3 Comportamiento y pruebas integradas

Caso	Entrada	Ruta esperada	Estado esperado
C1 fuera de alcance	«¿Cuál es el horario de la cafetería?»	RAG	evidencia insuficiente, sin fuentes
C2 ambigua	«¿Cómo lo hago?»	aclaración	aclaración
C3 sin código	«Dime el estado de mi solicitud»	aclaración	aclaración

Además se repiten los cinco casos iniciales (normal, ambiguo, incompleto, malicioso, fuera de alcance) más herramienta (código existente e inexistente) y revisión humana; el notebook verifica la ruta de cada uno. La lógica completa se validó localmente con componentes simulados (todas las rutas pasaron); la validación con los modelos reales se hace en Colab.

7.4 Revisión humana

El equipo lee cada respuesta generada (11 preguntas) y marca correcta, fiel al contexto, citas que respaldan, estado adecuado con una justificación breve en `REVISION_HUMANA`. Resultado a reportar: ____ / 11 en cada criterio.

8. Decisión técnica, fallo observado y límites

Decisión técnica. Se compararon tres recuperadores con las mismas preguntas. Criterio fijado antes de mirar la reserva: mayor MRR, luego mayor Recall@3, luego la opción más simple. Motivos para probar el híbrido: los manuales tienen términos exactos («403», «SOL-1001») que BM25 captura, mientras que los embeddings capturan paráfrasis («contraseña» frente a «Recuperar acceso»). El reranker agrega costo de latencia y un segundo modelo; solo se adopta si mejora de forma clara.

Fallo real observado (medido). Con búsqueda léxica sola, la pregunta R3 «Se me olvidó la contraseña, ¿cómo ingreso de nuevo?» recupera `tipos_solicitud-02`, `constancias-03` y `constancias-02`, pero no `acceso-01` (Recuperar acceso). Causa: desajuste de vocabulario; el manual dice «Recuperar acceso» y «no puede ingresar», no «contraseña». Qué se hace: la rama semántica (embeddings) debería cubrirlo; si en Colab A/B/C también fallan, la acción correcta es enriquecer el documento (sinónimos en el texto fuente) y registrarlo en la bitácora, no ajustar el sistema a esa pregunta de reserva. Completa en la bitácora si el fallo persiste con los embeddings reales.

Limitaciones.

Límite	Consecuencia
Corpus de 5 documentos / 12 fragmentos	Las diferencias A/B/C pueden ser ruido; no generaliza a manuales grandes.
11 preguntas anotadas por el mismo equipo	Sesgo posible; sin segundo anotador.
Umbral de abstención 0,30 heurístico	Puede rechazar preguntas válidas o aceptar fuera de alcance; se verifica con C1 y se calibra.
Seguridad por patrones	No detiene toda inyección; se mitiga con herramienta de solo lectura y respuestas ancladas al contexto.
Herramienta sobre datos ficticios	No hay base real de solicitudes ni autenticación del solicitante.
Un LLM y un modelo de embeddings	Sin comparación entre modelos; sujetos a cambios del proveedor.
Sin pruebas de carga	Latencias medidas por consulta aislada, en CPU.

9. Privacidad y manejo de secretos

Todo el contenido es ficticio. Los códigos SOL-xxxx no corresponden a personas. Las claves viven solo en Colab Secrets (nunca en celdas, ni en el repositorio, ni en la evidencia: el notebook revisa el ZIP buscando patrones de claves y falla si encuentra alguno). Supabase usa RLS por propietario y la clave publishable, nunca la secret. Los logs enmascaran claves. En un uso real habría que añadir consentimiento, retención limitada de logs y autenticación del solicitante antes de mostrar estados.

10. Cómo ejecutar

Paso	Acción
1	Crear proyecto Supabase; ejecutar `supabase/setup.sql` en SQL Editor; crear un usuario con correo y contraseña.
2	En Colab > Secrets: SUPABASE_URL, SUPABASE_PUBLISHABLE_KEY, SUPABASE_EMAIL, SUPABASE_PASSWORD, GROQ_API_KEY (activar acceso al notebook).
3	Abrir `Gestor_Solicitudes_Consolidado.ipynb` y ejecutar todo. La primera vez construye el índice; las siguientes lo reutilizan.
4	Completar: tabla 7.2, `REVISION_HUMANA`, fallo real y bitácora. Descargar el ZIP de evidencia.

Paso	Acción
5	Subir al repositorio: README, notebook, SQL, corpus, requirements, bitácora y este PDF (sin secretos).

Modos: `modo_llm='mock'` permite probar clasificación y herramienta sin API (RAG extractivo, claramente marcado); `almacen='local'` es solo para pruebas y muestra una advertencia: la entrega final debe usar Supabase.

11. Checklist de entrega (los 12 ítems)

#	Ítem exigido	Dónde está	Estado
1	Función original + entrada del usuario	Notebook §5 `clasificar`, §10 `atender`	Listo
2	Integración con modelo y prompt RAG versionado	§1 `llamar_llm`, §9 `rag_v2`	Listo
3	Salida controlada (contrato, Pydantic, reglas)	§5, §9 + pruebas negativas	Listo
4	Herramienta integrada validada, solo lectura	§7	Listo
5	Seguridad y manejo de errores	§1, §8, §13 (falla simulada)	Listo
6	Pruebas, logs, bitácora (5 casos, latencia, tokens)	§13, `bitacora.csv`	Ejecutar en Colab
7	Corpus documentado	§2 + catálogo, sección 3 de esta guía	Listo
8	Chunking y metadatos	§3	Listo
9	Embeddings + Supabase (SQL, reutilización)	§4, `supabase/setup.sql`	Ejecutar en Colab
10	Recuperación, fuentes, enrutamiento RAG vs herramienta	§9–10	Listo
11	Mejora justificada A vs B/C	§11–12	Ejecutar en Colab
12	Evaluación integrada, revisión humana, fallo, límites	§12 + secciones 7–8	Completar con resultados

Repositorio: README · código/notebook · SQL de Supabase · dependencias · corpus · pruebas · bitácora · PDF · sin secretos. Nuevo notebook final que corre completo en Colab sin otros notebooks.

12. Pendientes que solo el equipo puede cerrar

• Ejecutar el notebook en Colab con sus credenciales y pegar la tabla A/B/C (sección 7.2).
• Rellenar la revisión humana y reemplazar el caso de la sección 6 por la salida real.
• Confirmar el fallo real con embeddings reales y anotarlo en la bitácora.
• Si el docente exige los manuales originales de la Clase 08 (acceso.md, constancias.md), reemplazar los textos del notebook conservando las frases anotadas o reanotar los casos: el chequeo automático de la sección 12 avisa si una frase ya no existe.